

PYTHON PROJECT

STUDENT EXPENSE TRACKER

SUBMITTED BY

Name	Arpita Sachan
Registration number	26BAI10229
Department	AI & ML
Institute	VIT BHOPAL UNIVERSITY
Academic year	2026 –27

SUBMITTED TO

Name	Venkat Prasad Padhy
Designation	Assistant professor
Department	CSE

2. INTRODUCTION

The **Student Expense Tracker** is a software application developed to help students manage and track their daily expenses in an easy and organized way. Students spend money on various activities such as food, transportation, education, shopping, entertainment, and other personal needs. Keeping track of all these expenses manually can be difficult and time-consuming.

The main purpose of the Student Expense Tracker is to provide a simple platform where students can record their expenses, categorize them, and monitor their spending habits. The system helps users understand how much money they spend and on which categories they spend the most.

By maintaining expense records digitally, students can easily view their previous transactions and manage their budget more effectively. The application also reduces the need for maintaining paper-based records and makes expense management faster and more convenient.

This project focuses on developing a user-friendly and efficient system that promotes better financial management among students. It also demonstrates the practical implementation of software development concepts such as data management, user interface design, CRUD operations, and system testing.

3. PROBLEM STATEMENT

Students often must manage their expenses within a limited budget. They spend money on different categories such as food, transportation, education, shopping, entertainment, and other daily activities. However, many students do not maintain proper records of their expenses, making it difficult to know where and how much money is being spent.

Manual expense tracking using notebooks or spreadsheets can be time-consuming and inconvenient. It may also lead to missing records, calculation errors, and difficulty in analysing spending patterns.

Therefore, there is a need for a simple and user-friendly **Student Expense Tracker** that allows students to record, categorize, monitor, and manage their daily expenses digitally. The system should provide an organized way to maintain expense records and help users understand their spending habits so that they can manage their budget more effectively.

4. FUNCTIONAL REQUIREMENTS

The Student Expense Tracker provides the following functional requirements:

1. **Add Expense**
The system allows the user to add a new expense by entering the required expense details.
2. **View All Expenses**
The system displays all the expenses recorded by the user.
3. **Search Expenses**
The system allows the user to search for a specific expense from the available expense records.
4. **Total Expense**
The system calculates and displays the total amount of all recorded expenses.
5. **Highest Expense**
The system identifies and displays the expense with the highest amount.
6. **Delete Expense**
The system allows the user to delete an existing expense from the records.
7. **Save Expense**
The system saves the expense details so that the records can be maintained for future use.
8. **Main Menu**
The system provides a main menu through which the user can access all available expense management operations.

5. NONFUNCTIONAL REAUIREMENTS

Non-functional requirements describe the quality, performance, and usability of the Student Expense Tracker system.

1. **Usability**

The system should have a simple and easy-to-understand interface so that users can easily perform expense-related operations.

2. **Performance**

The system should respond quickly to user commands such as adding, searching, viewing, and deleting expenses.

3. **Reliability**

The system should perform the required operations correctly and maintain accurate expense records.

4. **Data Accuracy**

The system should correctly calculate the total expense and identify the highest expense without calculation errors.

5. **Maintainability**

The system should be easy to maintain, modify, and update whenever new features or changes are required.

6. **Security**

The system should protect stored expense information from unauthorized modification or deletion.

7. **Efficiency**

The system should use system resources efficiently and should be capable of handling the user's expense records without unnecessary processing.

8. **Portability**

The application should be capable of running on systems that support the technologies and environment used to develop the project.

6. SYSTEM ARCHITECTURE

The **Student Expense Tracker** is designed using a simple and modular architecture that allows the user to manage daily expenses efficiently. The system takes input from the user through the main menu, processes the selected operation, stores or retrieves expense data, and displays the appropriate result.

Architecture Components

The system consists of the following major components:

1. User

- a. The user interacts with the application through the main menu.
- b. The user can add, view, search, delete, and save expenses.
- c. The user can also calculate the total expense and find the highest expense.

2. Main Menu

- a. Acts as the entry point of the application.
- b. Displays all available operations to the user.
- c. Takes the user's choice and directs it to the appropriate function.

3. Expense Management Module

- a. Handles all expense-related operations.
- b. Includes:
 - i. Add Expense
 - ii. View All Expenses
 - iii. Search Expenses
 - iv. Total Expense
 - v. Highest Expense
 - vi. Delete Expense
 - vii. Save Expense

4. Processing Module

- a. Processes the selected operation.
- b. Performs calculations such as total expense and highest expense.
- c. Searches and identifies the required expense record.

5. Data Storage

- a. Stores expense records for future access.
- b. Provides data to the application whenever the user requests to view, search, calculate, or delete expenses.

6. output

- a. Displays the result of the selected operation to the user.
- b. Shows expense records, calculated values, search results, and confirmation messages.

7. DESIGN DIAGRAMS



8. DESIGN DECISION & RATIONAL

1. Python Programming Language

Design Decision: Python is used to develop the Student Expense Tracker.

Rationale: Python has simple and readable syntax, making it suitable for developing a beginner-friendly expense management application.

2. List-Based Data Storage

Design Decision: A Python list is used to store expense records.

Rationale: Lists are simple to use and allow multiple expenses to be stored and processed during program execution.

3. Function-Based Design

Design Decision: Different operations are implemented using separate functions, such as adding, viewing, searching, and calculating expenses.

Rationale: Functions make the program organized, reusable, and easier to understand, test, and modify.

4. Menu-Driven Interface

Design Decision: A menu is provided so users can select different expense-management operations.

Rationale: A menu-driven interface makes the application easy for students to use and navigate.

5. User Input

Design Decision: The application accepts expense details such as category, amount, and description from the user.

Rationale: Taking input from the user makes the tracker interactive and allows students to record their personal expenses.

6. Expense Categories

Design Decision: Expenses are organized using categories such as Food, Travel, Education,

9. IMPLEMENTATION DETAILS

9.1 Programming Language and Tools

The Student Expense Tracker is implemented using **Python**. The project is developed as a beginner-friendly command-line application. **GitHub** is used for storing and managing the source code and maintaining version history.

9.2 Main Program Implementation

The main functionality of the project is implemented in `main.py`. The program provides a menu-driven interface through which the user can manage daily expenses.

The major operations implemented in the program are:

- Adding a new expense
- Viewing recorded expenses
- Calculating total spending
- Organising expenses according to categories
- Managing expense records through menu options
- Displaying appropriate messages to the user

9.3 User Input and Processing

The program accepts required information from the user through the command-line interface. The entered expense information is processed according to the selected menu option. The program uses Python variables, conditional statements, loops, lists/data structures, and functions wherever required to perform the operations.

9.4 Validation and Error Handling

Basic validation is incorporated to handle incorrect or unexpected user inputs. The program displays appropriate messages when an invalid menu option or unsuitable input is entered, helping the user operate the application correctly.

9.5 Menu-Driven Interface

The application follows a menu-driven approach. When the program starts, the user is presented with different choices. The selected option determines which operation is performed. After completing an operation, the user can continue using other available options or exit the program.

9.6 Testing and Output

The project was tested by executing different menu options and checking the resulting outputs. Screenshots of the program execution have been added to the GitHub repository as evidence of testing and working functionality.

11. TESTING APPROACH

The Student Expense Tracker was tested to verify that all implemented functions work correctly and produce the expected results. Different valid and invalid inputs were used during testing, and the actual output was compared with the expected output.

11.1 Testing Method

The project was tested manually by running the Python program and checking each major functionality separately. The main functions of the system were executed with different inputs to verify their correctness.

11.2 Test Cases

Test Case	Functionality Tested	Input / Action	Expected Result
TC01	Add Expense	Enter valid expense details	Expense should be added successfully.
TC02	Search Expense	Enter a keyword or relevant expense detail	Matching expense should be displayed correctly.
TC03	Total Expense	Select the Total Expense option	Total amount of all expenses should be displayed correctly.
TC04	Highest Expense	Select the Highest Expense option	Expense with the highest amount should be displayed.
TC05	Delete Expense	Select an expense for deletion	Selected expense should be removed successfully.
TC06	Save Expense to File	Add or delete an expense and save the data	Updated expense data should be saved to the file.
TC07	Invalid Input	Enter an invalid value where applicable	Program should display an appropriate error or validation message.
TC08	Menu Navigation	Select different menu options	The corresponding functionality should execute correctly.

11.3 Testing Result

The testing process verified that the implemented functions of the Student Expense Tracker work according to their intended purpose. The major operations such as

adding, searching, calculating total expenses, finding the highest expense, deleting expenses, and saving expense data to a file were tested.

11.4 Error Handling

Input validation and error handling were considered during testing to reduce incorrect inputs and prevent unexpected program behaviour. Invalid inputs were checked wherever validation was implemented in the project.

11.5 Conclusion

The testing process helped verify the functional behaviour of the Student Expense Tracker and identify issues during development. After testing and correcting the identified issues, the implemented operations were executed successfully.

12. CHALLENGES FACED

During the development of the Student Expense Tracker, several challenges were faced while implementing and testing the project.

12.1 Understanding the Project Structure

One of the initial challenges was organizing the project into separate files and modules. It required understanding how different components of the program work together.

12.2 Implementing Expense Operations

Implementing operations such as adding, searching, calculating total expenses, finding the highest expense, and deleting expenses required careful handling of expense data and program logic.

12.3 File Handling

Saving expense information to a file was another challenge. The program had to ensure that the expense data was stored correctly and could be accessed when required.

12.4 Input Validation

Handling incorrect or unexpected user inputs was challenging. Validation was required to reduce errors and ensure that the program behaved correctly during different inputs.

12.5 Testing and Debugging

During testing, some issues were identified in the program's execution. These issues were analysed and corrected by checking the program logic and testing the functions repeatedly.

12.6 GitHub and Project Management

Maintaining the project files in an organized GitHub repository and keeping the documentation updated was also part of the development process.

12.7 Overall Challenge

The main challenge was integrating all the individual functions into one working Student Expense Tracker while maintaining a clear and modular structure. Testing each feature separately helped in identifying and resolving issues during development.

13. Learnings & Key Takeaways

Developing the Student Expense Tracker provided practical experience in applying Python programming concepts to a real-world problem. The project helped in understanding how individual programming concepts can be combined to build a complete working application.

13.1 Python Programming

The project improved understanding of Python fundamentals, including variables, data types, conditional statements, loops, functions, and lists.

13.2 Modular Programming

The project provided experience in dividing the program into different modules and files. This helped in keeping the code organized, readable, and easier to maintain.

13.3 File Handling

The project provided practical knowledge of storing and retrieving expense information using file handling techniques.

13.4 Data Processing

The project helped in understanding how expense data can be processed to perform operations such as searching expenses, calculating total expenses, and finding the highest expense.

13.5 Input Validation and Error Handling

The project demonstrated the importance of validating user inputs and handling errors to improve the reliability of the application.

13.6 Testing and Debugging

Testing different functions helped in identifying errors and improving the overall functionality of the program. Debugging also improved the understanding of program execution and logic.

13.7 GitHub and Version Control

The project provided practical experience in using GitHub to store, organize, and manage project files and documentation.

13.8 Key Takeaway

The major learning from this project was understanding how programming concepts can be combined to develop a structured and useful application. The project also improved problem-solving, debugging, documentation, and project management skills.

14. Future Enhancements

The Student Expense Tracker can be further improved by adding more features and making the application more user-friendly and efficient.

14.1 Graphical User Interface

A graphical user interface can be added to make the application easier to use instead of relying only on the command-line interface.

14.2 Expense Categories

Expenses can be categorized into groups such as Food, Travel, Education, Shopping, and Entertainment for better organization.

14.3 Date-wise Expense Tracking

A date feature can be added to record and view expenses based on specific dates or time periods.

14.4 Budget Management

A budget feature can be introduced to allow users to set spending limits and monitor their expenses.

14.5 Expense Reports

The application can be enhanced to generate detailed expense reports and summaries for better financial analysis.

14.6 Data Export

An option to export expense data to formats such as CSV or Excel can be added for easier data management.

14.7 Data Visualization

Charts and graphs can be introduced to visually represent spending patterns and make expense analysis easier.

14.8 Web or Mobile Application

In the future, the project can be converted into a web or mobile application so that users can manage their expenses from different devices.

15. References

1. Python Software Foundation, Python Documentation.
2. VITyarthi, Build Your Own Project – Project Guidelines, VIT Bhopal University.
3. GitHub Documentation, GitHub Repository and Version Control.
4. Visual Studio Code Documentation, Microsoft.
5. Python File Handling Documentation, Python Software Foundation.